Faisal Bahadur
Asim Shahzad

GREEDYBINNER: A RESOURCE-BOUNDED ADAPTIVE HISTOGRAM FOR INTERPRETABLE STREAMING LATENCY QUANTILE ESTIMATION

Abstract *In the field of quantile estimation for continuously arriving data streams, the two mainstream solutions have inherent limitations: sketch-based methods feature compact storage and fast computation, but their results are difficult to interpret; histogram methods have strong readability, but suffer from poor bin adaptability, and easily lose detailed information of skewed data. The proposed framework, GreedyBinner, addresses both limitations through adaptive binning with boundary constraints. Its main algorithmic contribution is that it combines width-constrained adjacent-bin extension with count-prioritized greedy merging under a fixed bin budget. Unlike fixed-layout or log-spaced structures, GreedyBinner can adjust bin boundaries during ingestion; unlike sketch-only methods, it keeps explicit bin ranges for interpretation. New values are either counted inside an existing bin, absorbed by a neighboring bin when the width limit allows it, or placed into a new unit bin [floor(value), floor(value)+1). A separate display step converts the adaptive bins into readable histogram ranges without changing the quantile values. The method was evaluated on a skewed real-world latency stream and compared with t-digest, DDSketch, DynaHist, HDRHistogram, and BigML Histogram. GreedyBinner achieved the lowest relative error at P50 and P90 and matched the best result at P99. Its ingestion and query speeds were lower than the fastest methods, but remained practical for single-node latency monitoring, with sub-microsecond query latency and low JVM resource use.*

Keywords Streaming Quantile Estimation; Skewed Data Streams; Adaptive Histograms; Adaptive Binning; Resource-Bounded Algorithms; Interpretable Histograms

Copyright © 2026 Author(s). The paper has been accepted for publication in Computer Science journal: `http://journals.agh.edu.pl/csci`.

1. Introduction

Contemporary computing systems continuously generate large volumes of performance measurements, logs, and telemetry data [16]. These streams can add up to millions of events per second, even before any distributed system scaling is applied at the level of one server, database instance, or workstation. Such single-machine settings arise in local streaming-analysis agents, database-instance telemetry, workstation-level streaming latency tracking, edge gateways, IoT monitoring nodes, trading terminals, and single-server dashboards. In these cases, the main requirement is not to merge summaries from thousands of distributed nodes, but to obtain accurate percentiles, bounded memory use, and readable distribution summaries from one continuously evolving stream.

In monitoring systems, operators often need P50, P90, and P99 values rather than only the average, because these quantiles show latency behavior more clearly [5]. Exact quantile computation on large or continuous streams requires extra time and memory. Sketch-based estimators are commonly used because exact quantile computation is costly on continuous streams. Many of these methods use histogram-like summaries [1] but sketches are mainly designed for compact approximate quantile estimation and for distributed use, where summaries from many nodes must be merged [9]. On the other hand, the requirements of a single-node computing scenario are different (including bounded memory, fast updates, accurate P50/P90/P99 estimates, and readable output) where the situation becomes more complex in case of skewed or heavy-tailed latency streams [18], because such streams can reduce tail-estimation accuracy [3, 12], which may affect anomaly detection or service-level decisions.

In addition to percentiles, operators need visibility into the overall data distribution, which motivates the use of histogram-based summaries that approximate the data distribution using bucket summaries and help with visual interpretation [15]. Besides this, an improper bin setting will increase the difficulty of data quality inspection and distribution interpretation [4]. To solve the issue of clear views of data, schemes have been developed that are dependent on external entities to work with and do not suit single-node settings [22, 19].

To address these gaps, this study proposes GreedyBinner. It is a bounded adaptive histogram for estimating quantiles and visualizing skewed streams in single-node systems. GreedyBinner is designed as a resource-bounded adaptive histogram framework for streaming quantile estimation, where low-latency quantile estimation and interpretable distribution summaries are required under non-stationary data streams. This framework addresses these issues by providing consistent quantile estimation in single-machine settings and robustly dealing with strongly skewed data distributions. The quantile estimates are highly suitable for visualization and can be computed efficiently within bounded memory resources. The greedy binning approach allows us to have a strict limit on the number of active bins and provides predictable use of resources with good accuracy for key percentiles. The bin widths are also controlled for both normal and relaxed merges. Importantly, such accuracy is maintained for significantly skewed data streams, unlike several sketch-based baselines in the evaluated setting; GreedyBinner achieves the lowest error at P50 and P90 and matches the best-performing method at P99 in the evaluated stream.

Furthermore, an integrated visualization module transforms GreedyBinner's adaptive bins into human-interpretable intervals based on a uniform grid, making the output more suitable for readable histogram interpretation.

The primary insight is to combine three novel features into a resource-constrained streaming histogram: (1) a width-constrained greedy bin merger during data ingestion; (2) an adaptive greedy merger of bins based on their count, which ensures we respect our strict constraint on the number of active bins; and (3) a distinct visual-

isation format for presenting the histogram to a user. While existing methods offer some of these features (e.g. adaptive histogram bounds, succinct quantile sketches, visualisation for histograms), they do not combine these features together. While the constituent parts are not novel in isolation, the key insight is to design them together to provide a streaming quantile estimator with interpretable monitoring of the distribution using a fixed number of bins.

This paper is structured as follows: Section 2 and 3 present the related work and materials and methods. Section 4 and 5 report the evaluation and conclusion.

2. Related Work

The related work is grouped into three technical directions: histogram-based quantile estimators, quantile sketches, and visualization/oracle-based tracking methods.

2.1. Histogram-Based Quantile Estimators

DynaHist [7] updates bin boundaries as new observations arrive in order to reduce quantile estimation errors. It also keeps memory usage low, which makes it suitable for high-throughput data streams. In practice, however, it still depends on user-defined parameters such as the number of bins. Its accuracy can also decline when distributions are highly skewed or when the underlying stream changes faster than the histogram can adapt.

Circllhist [11] is widely used for recording and analyzing latency measurements that span a large dynamic range. Its configurable precision and logarithmic binning allow values that differ by several orders of magnitude to be represented compactly. It can therefore be used to monitor very high percentiles such as P99.

HdrHistogram [20] is a popular log-linear histogram technique for recording and analyzing high-dynamic-range latency measurements with configurable precision. Its fixed logarithmic binning compactly represents values that differ by several orders of magnitude, making it useful for extreme percentiles such as P99 and above. With suitable range and precision settings, HDRHistogram provides good tail accuracy with low memory use, but its fixed bin layout may lose local detail when the stream changes or values concentrate near the median.

The BigML histogram, proposed in reference [2], centers on the core features of interpretability, readable output, and online updates. According to the official BigML histogram documentation, its quantile estimation can be imprecise due to a fixed number of bins and the necessity of summarizing bins when depth is reached. Next, performance is dependent on configuration and interpretability of bins is restricted since merged bins may not correspond to real data boundaries, resulting in distributions that are difficult to analyse with certainty.

2.2. Quantile Sketches

t-digest [6] has been a popular choice of data structure for quantile estimation. Its mergeable and memory-friendly structure makes it suitable for scalable and distributed environments. Yet, t-digest lacks formal worst-case error bounds and, indeed, it has been shown that the empirical accuracy of t-digest degenerates for highly skewed or heavy-tailed datasets (and is poor even up to extreme percentiles). Additionally, although it is designed for compact percentile estimation, it does not offer support for generating histogram-style representations directly.

DDSketch [13] provides fixed relative-error bounds on quantiles by using log-spaced bins. It is also truly mergeable and streaming capable. Like t-digest, it is concerned only with quantile accuracy; it does not produce intuitive bin ranges for visualization. Building on top of DDSketch, later work proposed UDDSketch [8], a

merge-efficient and theoretically improved extension to it which offers broader range guarantees. Although conceptually useful, it is primarily oriented toward distributed systems. UDDSketch enhances DDSketch by providing a merge-efficient protocol for fusing sketch summaries in parallel or distributed environments while maintaining both size and error guarantees. However, while UDDSketch improves relative error, it outputs a sketch itself (not a histogram structure) and does not generate useful bin boundaries or visualization-ready outputs.

2.3. Visualization and Oracle-Based Quantile Tracking

The heatmap method introduced in [17] can intuitively present large-scale cloud monitoring data, but it does not involve quantile estimators or adaptive histogram methods; similarly, [10] relies on an external oracle to track quantiles and cannot be adapted to autonomous single-node scenarios. It also does not provide readable histogram bins. Therefore, these methods address useful parts of the problem, but they do not offer a standalone histogram-based quantile algorithm that combines bounded memory, local accuracy, and interpretable bin output.

2.4. Research Gap

Table 1 provides a consolidated comparison between the literature reviewed and GreedyBinner based on features such as Adaptive Bin Boundaries (ABB), Quantile Estimation (QE), Bounded Representation (BR), Readable Histogram Output (RHO), Distributed (D) or not, Hard Bin Budget Constraint (HBBC), and Count Prioritised Adjacent Merging (CPAM).

Table 1 illustrates that current approaches only address subsets of the desired functionality. Sketch-based methods mostly focus on providing efficient quantile estimates, whilst histogram-based methods allow for human-readable summaries of the underlying distribution. However, they differ in their adaptability and bin control. GreedyBinner solves this issue by providing an integrated solution to dynamically control the bins, merge based on priority of counts and provide human-readable histograms within a single streaming structure.

Table 1

Comparison of all schemes w.r.t. different features: adaptive bin boundaries (ABB), quantile estimation (QE), bounded representation (BR), readable histogram output (RHO), distributed (D) or not, hard bin-budget constraint (HBBC), count-prioritized adjacent merging (CPAM).

Approach	ABB	QE	BR	RHO	Distributed	HBBC	CPAM
DynaHist	Yes	Yes	Yes	Yes	No	No	No
Circllhist	No	Yes	Yes	Yes	No	No	No
HdrHistogram	No	Yes	Yes	Yes	No	No	No
BigML	Yes	Yes	Yes	Yes	No	No	No
t-digest	Yes	Yes	Yes	No	Yes	No	No
DDSketch	No	Yes	Yes	No	Yes	No	No
UDDSketch	No	Yes	Yes	No	Yes	No	No
CloudHeatMap	No	No	No	Yes	Yes	No	No
Oracle-Based	No	Yes	No	No	No	No	No
GreedyBinner	Yes	Yes	Yes	Yes	No	Yes	Yes

3. Design of GreedyBinner

We designed the proposed scheme in such a way that it can represent the incoming performance-monitoring stream through a small, ordered set of non-overlapping bins where each bin records its lower boundary, its upper-exclusive boundary, and the

number of values assigned to it. The algorithmic framework originally developed for this study has its main control parameters recorded in Table 2 and its overall workflow summarized in Figure 1. The first step of its logic is to check whether a newly input value is covered by any existing bin; if this condition is met, only the count of the corresponding bin is incremented. If not, GreedyBinner checks whether a left or right neighboring bin can be extended without crossing `MAX_WIDTH`. If it cannot be extended then a new single unit bin [floor(value), floor(value)+1) is inserted into the correct position. When there are more bins than maxBins then merging begins. Strict merging is tried first, relaxed merging second, and forced merging only at the end. The algorithm prioritizes the merging of bins with small counts. This ensures that the sparse areas of the data stream are compressed first and hence the high density regions will compress last, impacting the percentile estimates less.

This means GreedyBinner can be used to estimate quantiles on skewed streams, with constraints on memory usage, accuracy of percentiles and interpretability.

In the following subsections we explain the formulation and operation of the algorithm in detail.

Table 2
Main parameters used in GreedyBinner.

Parameter	Short description
bins: List	Stores the active BinEntry objects. Each bin keeps its lower bound, upper-exclusive bound, and count.
`MAX_WIDTH`	Preferred maximum width of a bin. It is used during bin extension and strict merging.
`maxBins`	Fixed upper limit on active bins. Merging starts when this limit is exceeded.
`MERGE_GAP_TOLERANCE`	Allowed empty gap between two nearby bins in the relaxed merge pass.
`WIDTH_TOLERANCE`	Extra width allowed in relaxed merging when strict merging cannot bring the bin count down.

The default parameters are `MAX_WIDTH`=10, `MERGE_GAP_TOLERANCE`=2, `WIDTH_TOLERANCE`=2 and `maxBins`=60. This provides a good balance between having tightly packed bins, some flexibility to merge bins and retaining detail in the distribution. The `MAX_WIDTH` parameter will help stop the bins getting too large. Having a larger `MAX_WIDTH` and/or `WIDTH_TOLERANCE` will cause the bins to be larger and there will be less merging. However, there will be less detail in the percentiles. Having smaller values will cause greater compression, but could cause more error in the quantiles. The `MERGE_GAP_TOLERANCE` parameter will mainly affect the sparse areas when you are performing a relaxed merge. The `maxBins` parameter will dictate how much compression occurs. A larger value will retain more detail, whereas a smaller value will cause more forced merging.

3.1. Formalization of GreedyBinner

Let the incoming data stream be represented as $X = \{x_1, x_2, \ldots, x_t\}$, where x_t is the value received at time t. At any time t, GreedyBinner stores the processed stream as a small, ordered set of non-overlapping bins:

$$B_t = \{b_1, b_2, \ldots, b_m\}, \qquad b_i = ([l_i, u_i), c_i). \tag{1}$$

Here, l_i is the lower boundary, u_i is the upper-exclusive boundary, and c_i is the number of values assigned to bin b_i. The bins are kept in sorted order, so consecutive

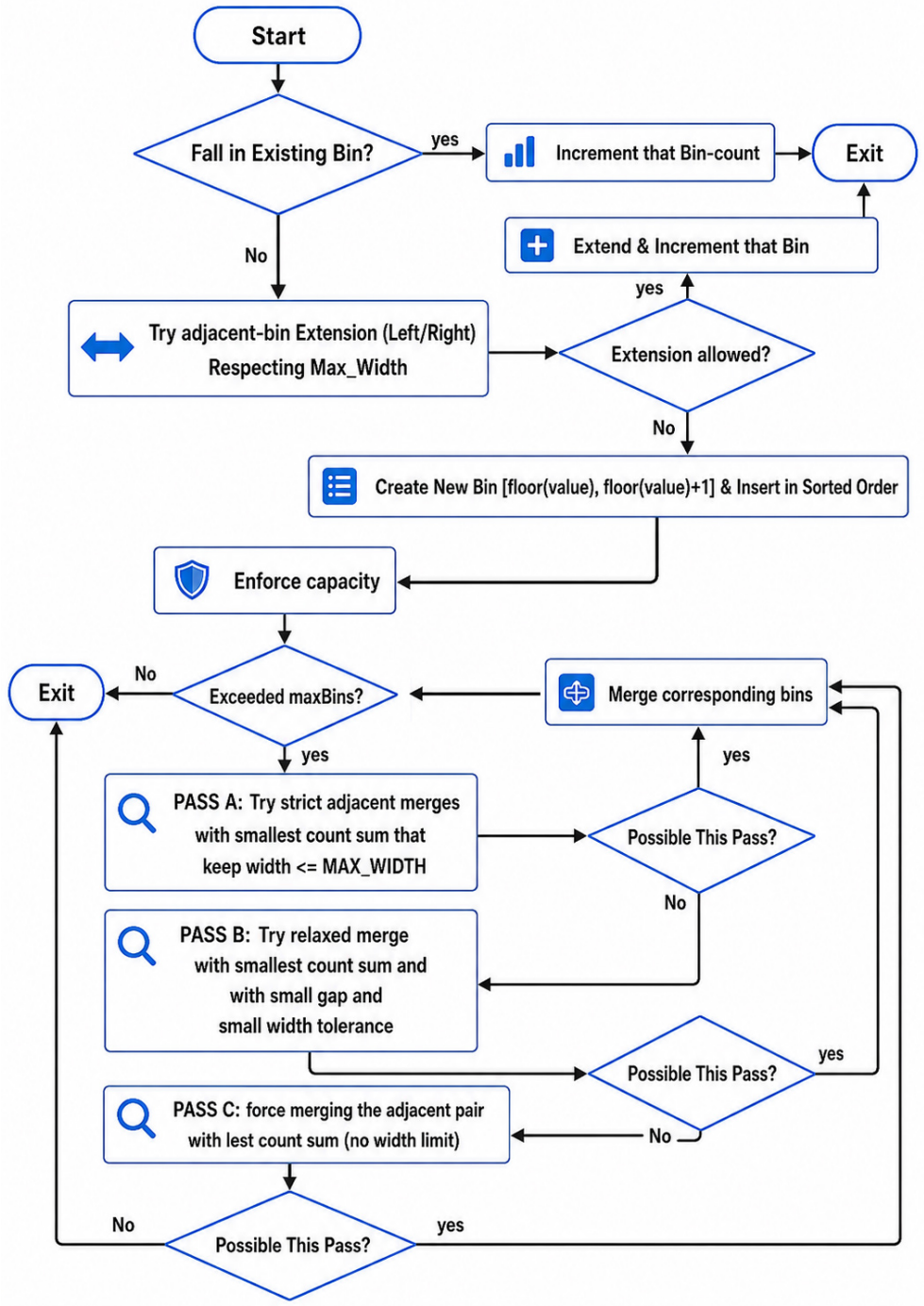


Figure 1. GreedyBinner's operational flow, showing value ingestion, bin updates, and the adaptive merge process used to maintain the fixed bin budget.

bins satisfy $u_i \leq l_{i+1}$. The total count is preserved, i.e., $\sum_{i=1}^{m} c_i = t$, which means that bin extension and merging do not discard observed values.

For each new value x_t, GreedyBinner first maps it to a unit interval $C(x_t) = [\lfloor x_t \rfloor, \lfloor x_t \rfloor + 1)$. If the value already falls inside an existing bin, only the corresponding count is increased. If not, GreedyBinner checks whether the unit interval can be absorbed by a neighboring bin without violating the preferred maximum width $W_{\max}$. The right-side and left-side extension conditions are:

$$\lfloor x_t \rfloor = u_i \quad \text{and} \quad (\lfloor x_t \rfloor + 1) - l_i \leq W_{\max}, \tag{2a}$$

$$\lfloor x_t \rfloor + 1 = l_i \quad \text{and} \quad u_i - \lfloor x_t \rfloor \leq W_{\max}. \tag{2b}$$

Equations (2a) and (2b) formalize the first novelty point of GreedyBinner. Here, bin edges can be dynamically expanded if the value can be contained within a local bin and the new width of bin is less than the desired size.

Otherwise, a new bin will be created with unit width $b_{\text{new}} = ([\lfloor x_t \rfloor, \lfloor x_t \rfloor + 1), 1)$. This new bin will be added to the list of sorted bins. If the amount of active bins exceeds the specified budget, we force the hard constraint on the number of bins as:

$$|B_t| \leq B_{\max}. \tag{3}$$

For two adjacent bins b_i and b_{i+1}, the merge operation is defined as:

$$b_i \oplus b_{i+1} = ([l_i, u_{i+1}), c_i + c_{i+1}). \tag{4}$$

GreedyBinner only merges neighboring bins to preserve the structure of the ordered histogram.

In the strict merge pass, GreedyBinner considers only adjacent pairs whose merged width remains within $W_{\max}$. The selected pair is the one with the smallest combined count:

$$\begin{aligned} i^* &= \arg\min_{i \in \mathcal{A}_s} (c_i + c_{i+1}), \\ \mathcal{A}_s &= \{i \mid u_{i+1} - l_i \leq W_{\max}\}. \end{aligned} \tag{5}$$

If no strict merge is possible, a relaxed pass allows a small gap G and an additional width tolerance τ:

$$\begin{aligned} i^* &= \arg\min_{i \in \mathcal{A}_r} (c_i + c_{i+1}), \\ \mathcal{A}_r &= \{i \mid l_{i+1} - u_i \leq G \text{ and } u_{i+1} - l_i \leq W_{\max} + \tau\}. \end{aligned} \tag{6}$$

If neither strict nor relaxed merging can restore the bin budget, GreedyBinner applies a forced merge by selecting the adjacent pair with the smallest combined count, without applying the width rule.

Equations (3)–(6) formalize the second novelty point of GreedyBinner. The method maintains a fixed bin budget by merging low-count neighboring regions first. Sparse regions are therefore compressed earlier, while denser regions are more likely to retain finer detail.

Quantile estimation is performed from cumulative bin counts. Let $N = \sum_{i=1}^{m} c_i$ be the total number of processed values. For a required quantile level q, where $0 < q < 1$, the target rank is $r_q = \lceil qN \rceil$. The bin containing this rank is the first bin b_j for which $\sum_{i=1}^{j} c_i \geq r_q$. If linear interpolation inside the selected bin is used, the estimated quantile is:

$$\widehat{Q}(q) = l_j + \frac{r_q - \sum_{i=1}^{j-1} c_i}{c_j}(u_j - l_j). \tag{7}$$

GreedyBinner is an algorithm that estimates percentiles with range-constrained bins. Internal bins can be easily mapped to a range of display bins, e.g 1-2-5 scale. Redistributing counts based on overlapping bins will cause the data to be shown in different ranges but the quantiles remain the same.

3.2. `ingest(value)`

This function that is provided as Algorithm 1 below describes the process of adding a new value into the GreedyBinner. For every new value, we map it to its unit cell [floor(value), floor(value)+1). It then checks whether the value already lies inside an existing bin, as represented in Eq. (1). If yes, only that bin's count is increased, and the bin boundaries remain unchanged.

If no existing bin contains the value, the algorithm maps the incoming value to its containing unit cell [floor(value), floor(value)+1). The algorithm then checks whether this unit cell is immediately adjacent to an existing bin. A right extension is possible when the cell lower bound equals the current upper-exclusive boundary of a bin as in Eq. (2a), and a left extension is possible when the cell upper bound equals the current lower boundary of a bin as in Eq. (2b). If the resulting width remains within `MAX_WIDTH`, the bin boundary is updated, and the bin count is incremented.

If an extension occurs and the new bin-width does not exceed the specified maximum, the bin is extended, and its count is incremented while preserving local data density in dense regions.

If neither containment nor extension is feasible, a new unit-width bin [floor(value), floor(value)+1) is created so that the incoming value is always contained within the newly created bin. The new bin is inserted in sorted order. After this step, `enforceAdaptiveBinControl()` is called if the number of bins exceed `maxBins` as in Eq. (3). Merging is therefore done only when the bin limit is crossed. Otherwise, `ingest()` ends after updating, extending, or inserting a bin.

3.3. `enforceAdaptiveBinControl()`

Algorithm 2 is used when the bin count goes above `maxBins` limit and keeps merging neighboring bins by three sequential phases until this limit is restored. The `enforceAdaptiveBinControl()` enforces compactness through a hierarchy of merging passes, where strict merges preserve bin-width constraints, relaxed merges allow small gaps and forced merges target the low count bins. Thus, `maxBins` remain the hard limit (Eq. (3)). `MAX_WIDTH` is only the preferred width rule. Each pass scans the active bin list, so one pass costs $O(b)$. In the worst case, repeated merges can cost $O(b^2)$, but b is bounded by `maxBins`.

In normal runs, the forced merge is only a fallback; its main purpose is to keep the bin budget fixed.

GreedyBinner only enforces a maximum limit on the number of bins. However, `MAX_WIDTH` will be used when extending and enforcing strict merges. During the relaxed phase there is some tolerance. If `maxBins` cannot be achieved then it may exceed the `MAX_WIDTH` limit. In this case the pair with the lowest joint count will be merged to ensure we have a fixed budget of bins. This will primarily affect regions with sparse data.

3.4. Visualization & Interpretability

In practice, a quantile summary is more useful when the same output can also be read on a dashboard. To improve visualization, we created a lightweight utility DisplayBinsUtil. It changes only the displayed bin ranges, not the quantile values. Internal

Algorithm 1 GreedyBinner.ingest(value)

Require: value (double)
Ensure: bins updated after value insertion

Constants: `MAX_WIDTH`, `maxBins`
cellLower $\leftarrow \lfloor$ value $\rfloor$
cellUpper $\leftarrow$ cellLower $+1$
for all bin in bins **do**
    if bin.contains(value) **then**
        bin.increment()
        return
    end if
end for
for all bin in bins **do**
    $L \leftarrow$ bin.getLower()
    $U_{\text{ex}} \leftarrow$ bin.getUpperExclusive()
    if cellLower $= U_{\text{ex}}$ **then**
        newWidth $\leftarrow$ cellUpper $- L$
        if newWidth $\leq$ `MAX_WIDTH` **then**
            bin.setUpperExclusive(cellUpper)
            bin.increment()
            return
        end if
    end if
    if cellUpper $= L$ **then**
        newWidth $\leftarrow U_{\text{ex}} -$ cellLower
        if newWidth $\leq$ `MAX_WIDTH` **then**
            bin.setLower(cellLower)
            bin.increment()
            return
        end if
    end if
end for
newBin $\leftarrow$ BinEntry(cellLower, cellUpper)
bins.add(newBin)
bins.sort(by lower bound)
if bins.size() $>$ `maxBins` **then**
    enforceAdaptiveBinControl()
end if

bins are created for quantile estimation, display bins are created solely for displaying data so changing display bins will not affect the structure of the quantile estimation. This utility transforms raw adaptive bins into evenly spaced display bins aligned to a 1–2–5 grid step (e.g., 1, 2, 5, 10, ...). It selects a suitable step size based on the span of data and target number of displayed bins. Next, it adjusts the bin factor to meet this step, giving human-readable bin ranges.

While the display operation does not affect the number of values in the data, it can cause a loss of local information as the data values are mapped onto a larger scale during the process. The resulting visualisation is therefore primarily for readability.

DisplayBinsUtil is treated as part of GreedyBinner's native dashboard-output pipeline rather than as an external post-processing step. In the evaluation, competing methods are therefore visualized using their own native or directly exposed bin structures, while GreedyBinner is visualized using the display representation produced by its built-in utility.

Algorithm 2 GreedyBinner.enforceAdaptiveBinControl()

Require: Current ordered list bins; parameter `maxBins`
Ensure: Bins merged until bins.size() $\leq$ `maxBins` subject to pass order

if bins.size() $\leq$ `maxBins` **then** ▷ Eq. (3)
  return
end if
while bins.size() $>$ `maxBins` **do** ▷ Eq. (3)
  bestIdx $\leftarrow -1$
  bestCountSum $\leftarrow +\infty$
  for all adjacent pair (b_1, b_2) in bins **do**
    mergedWidth $\leftarrow b_2$.getUpperExclusive() $-b_1$.getLower()
    if mergedWidth $\leq$ `MAX_WIDTH` **then** ▷ Eq. (5)
      countSum $\leftarrow b_1$.getCount() $+b_2$.getCount()
      if countSum $<$ bestCountSum **then** ▷ Eq. (5)
        bestCountSum $\leftarrow$ countSum
        bestIdx $\leftarrow$ index of b_1
      end if
    end if
  end for
  if bestIdx $\neq -1$ **then**
    mergeBins(bestIdx,bestIdx+1) ▷ Eq. (4)
    continue
  end if
  bestIdx $\leftarrow -1$
  bestCountSum $\leftarrow +\infty$
  for all adjacent pair (b_1, b_2) in bins **do**
    gap $\leftarrow b_2$.getLower() $-b_1$.getUpperExclusive()
    mergedWidth $\leftarrow b_2$.getUpperExclusive() $-b_1$.getLower()
    if gap $\leq$ `MERGE_GAP_TOLERANCE` **then**
      if mergedWidth $\leq$ `MAX_WIDTH` + `WIDTH_TOLERANCE` **then** ▷ Eq. (6)
        countSum $\leftarrow b_1$.getCount() $+b_2$.getCount()
        if countSum $<$ bestCountSum **then** ▷ Eq. (6)
          bestCountSum $\leftarrow$ countSum
          bestIdx $\leftarrow$ index of b_1
        end if
      end if
    end if
  end for
  if bestIdx $\neq -1$ **then**
    mergeBins(bestIdx,bestIdx+1) ▷ Eq. (4)
    continue
  end if
  bestIdx $\leftarrow -1$
  bestCountSum $\leftarrow +\infty$
  for all adjacent pair (b_1, b_2) in bins **do**
    countSum $\leftarrow b_1$.getCount() $+b_2$.getCount()
    if countSum $<$ bestCountSum **then**
      bestCountSum $\leftarrow$ countSum
      bestIdx $\leftarrow$ index of b_1
    end if
  end for
  if bestIdx $\neq -1$ **then**
    mergeBins(bestIdx,bestIdx+1) ▷ Eq. (4)
    continue
  end if
  break
end while

3.5. Complexity of GreedyBinner

The cost of GreedyBinner mainly comes from ingestion and adaptive bin control. For each new value, the algorithm scans the active bins to check containment or adjacency. This scan costs $O(b)$, where b is the current number of active bins. Since b is bounded by the configured limit $B_{\max}$, the cost does not grow with the total stream length. Thus, the per-ingestion complexity is:

$$T_{\text{ingest}}(n) = O(B_{\max}). \tag{8}$$

where $B_{\max}$ is typically small (e.g., 60), ensuring efficient ingestion. Sorting may be needed after some bin updates, but its cost is limited because it depends only on the bounded bin list:

$$T_{\text{sort}} = O(B_{\max} \log B_{\max}). \tag{9}$$

which remains insignificant for small $B_{\max}$.

Adaptive merging is triggered only when the bin count temporarily exceeds $B_{\max}$ by a small constant factor. In such cases, the `enforceAdaptiveBinControl()` method performs a limited number of merging passes, each costing at most linear time in the number of bins:

$$T_{\text{merge}} = O(B_{\max}). \tag{10}$$

Merging is not done on every input value. It starts only when a new bin pushes the count above $B_{\max}$. The extra cost is therefore occasional and remains tied to the fixed bin list. The overall per-element runtime can be written as:

$$T(n) = O(B_{\max} + B_{\max} \log B_{\max}). \tag{11}$$

which simplifies to:

$$T(n) = O(B_{\max} \log B_{\max}). \tag{12}$$

given the dominance of the sorting term. Importantly, because $B_{\max}$ is a user-defined constant independent of input size n, the effective ingestion complexity reduces to:

$$T(n) \approx O(1). \tag{13}$$

in practice.

Thus, GreedyBinner achieves near-constant time ingestion per value, with bounded overhead from adaptive merging and negligible sorting cost while maintaining fidelity of the quantile-critical regions of the data stream. This ensures high efficiency even for large-scale streaming workloads.

The complexity results above describe why GreedyBinner is appropriate for continuous, low-latency streaming environments. As there are fixed number of bins, this algorithm has a bounded runtime for ingestion and merging. This makes it suitable for soft real time processing, rather than hard real time with guaranteed deadlines.

4. Evaluation

4.1. Experimental Plan

We evaluated GreedyBinner (GB) against five Java-based streaming quantile or histogram schemes: t-Digest (TD), DDSketch (DD), DynaHist (DH), HDRHistogram (HDR), and BMLHistogram (BML). The purpose of the evaluation was to test whether GB can serve as a bounded adaptive histogram method for streaming latency data under bounded resource conditions. The CloudHeatMap [17] was not tested since it has no estimation functionality, only visualization. Oracle based methods were not tested since they require prior knowledge of quantile values, which would not be possible in an autonomous stream setting. The Circllhist was not tested since no active Java implementation is available. Benchmarking across different languages may also introduce extra variance. Finally, the UDDSketch was not tested due to implementation issues and the fact that it focuses on distributed merge capabilities which is outside the scope of this work (single node).

All methods were evaluated using their standard or recommended configurations, while the dataset, Java runtime, ingestion schedule, sampling interval, and evaluation procedure were kept the same. The evaluation criteria included P50, P90, and P99

accuracy, ingestion speed, query latency, JVM heap and CPU usage, and histogram interpretability. Interpretability was assessed only for GB, DH, HDR, and BML because TD and DD do not directly expose simple human-readable histogram bins.

The experiments used the PlanetLab Round-Trip Time (RTT) dataset [23]. The PlanetLab trace, although containing data from many nodes, was treated as a single heavy-tailed latency trace. This allowed us to evaluate the performance of our methods on a single node. Furthermore, since the trace is realistic and skewed, we can also use it to test the quantile and histogram methods.

All schemes were tested in a streaming setup without full-data preloading. A lightweight producer–consumer framework replayed the PlanetLab traces [23] at 1,000 samples/s. The producer supplied fixed-size batches, while a semaphore synchronized ingestion so that all schemes received the same data under identical arrival conditions. Synthetic workloads were generated at the same rate using a timer-based generator without file I/O.

During each run, JVM heap and CPU usage were sampled every five seconds using JVisualVM [21] and a small JMX monitor [14]. Per-object profiling was avoided because it could add measurement overhead.

4.2. Percentile Accuracy and Speed Performance Tests

4.2.1. Percentile Accuracy Comparison

For the accuracy test, we compared the estimated and exact values of P50, P90, and P99. Relative error was calculated every five seconds during the 30-minute run, and the average error was then computed for each method. Figure 2 shows the relative-error traces for all schemes at P50, P90, and P99.

The plots illustrate the temporal stability and comparative accuracy of each method across low, mid, and high latency quantiles. Table 3 reports the corresponding average relative errors, which serve as the basis for the aggregated plots shown in Figure 3. Finally, Figure 3 visualizes the average relative-error values reported in Table 3, while Table 4 reports the corresponding standard deviations.

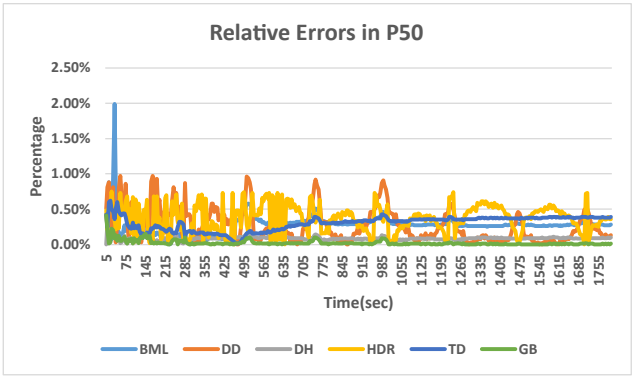
As shown in Table 3, GB consistently showed the lowest relative errors at P50 and P90 and matched the best-performing method at P99. This robustness originates from greedy bin-extension and adaptive bin-control. TD and DD are less accurate (Table 3), which is in line with earlier reports on skewed streams [3, 12]. HDR matches GB at P99 but is weaker at P50, while DH and BML remain in the middle range. We can observe the error variation in Table 4; DH is lowest at P50, with GB close. At P90, GB has the lowest variation. At P99, HDR is the most stable, followed by GB. Overall, GB remains either first or second in stability across P50, P90, and P99, which shows that its accuracy is steady during the streaming run.

GB in general provided better single-node accuracy and temporal stability at all tested quantiles while preserving the essential information content of the stream. Although it was not built for large-scale mergeable analytics, it is intended to be a lightweight adaptive histogram structure suitable for continuous, low-latency local monitoring under highly skewed workloads.

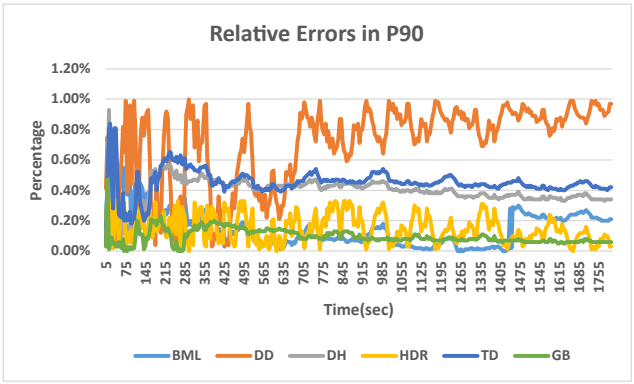
4.2.2. Comparative Analysis of Ingestion Speed

We quantified the number of values that each scheme could manage per second until it ingested the whole dataset (4.32 million RTT samples). Table 5 presents the throughput results, where HDR achieved the highest throughput (45.8 M values/s) as it has pre-allocated log-linear buckets and directly indexes memory.

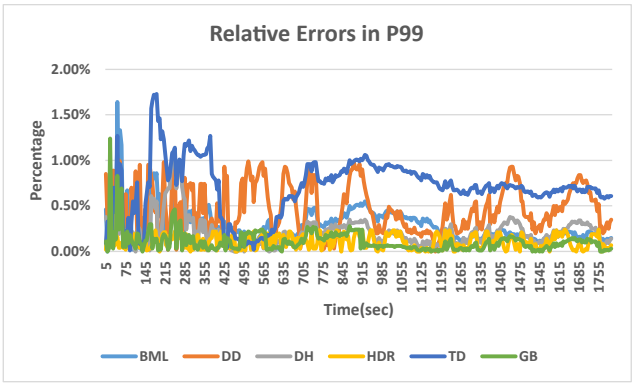
DH was in second position (42.6 M values/s), not too far behind, showing that its adaptive design introduces a minimal amount of overhead. DD reached 35 M values/s, throttled by logarithmic re-maps and internal reassignment. In Table 5, GB



(a)



(b)



(c)

Figure 2. Relative error series of all schemes over 30 minutes, shown separately for (a) P50, (b) P90, and (c) P99. The plots illustrate the temporal stability and comparative accuracy of each method across low, mid, and high latency quantiles.

Table 3

Average Relative Errors of all schemes over the 30-minute experiment

Scheme_Name	RE_P50_Avg	RE_P90_Avg	RE_P99_Avg
BML	0.27%	0.15%	0.28%
DD	0.27%	0.71%	0.46%
DH	0.09%	0.40%	0.22%
HDR	0.37%	0.15%	0.11%
TD	0.30%	0.45%	0.70%
GB	0.02%	0.10%	0.11%

Table 4
Standard Deviations of Relative Errors of all schemes over the 30-minute experiment

Scheme_Name	STD(RE_P50)	STD(RE_P90)	STD(RE_P99)
BML	0.001480183	0.001155421	0.001809263
DD	0.002352858	0.002639328	0.00265579
DH	0.000273542	0.000846687	0.001688562
HDR	0.001983208	0.000908814	0.000747497
TD	0.00101778	0.000839511	0.003071356
GB	0.000380352	0.000481537	0.001139902

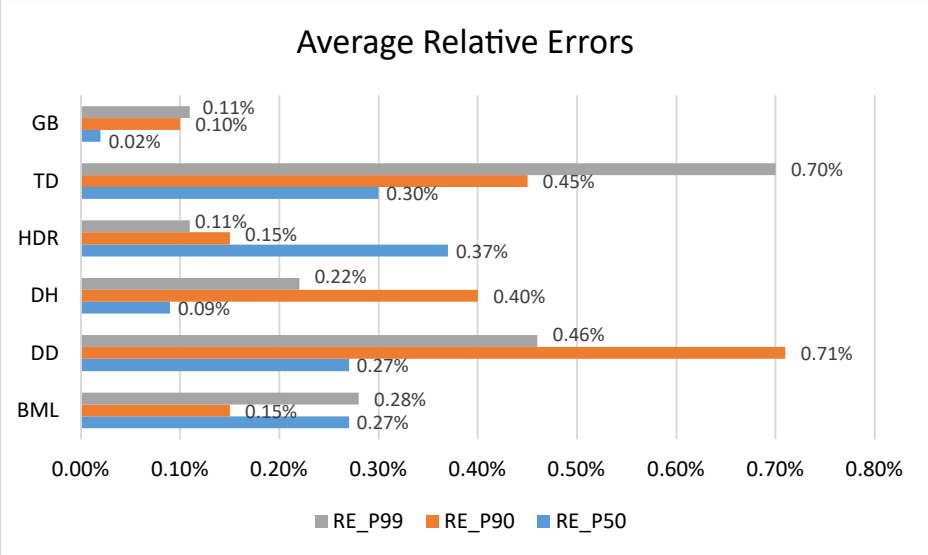


Figure 3. Aggregated comparison of average relative errors across all schemes for P50, P90, and P99, derived from the values reported in Table 3.

processed 13.4 M values/s which is slower than the fastest schemes but expected given its design emphasis on preserving critical information and maintaining high quantile accuracy. In the case of GB each update involves bin containment checks, adjacency-based extensions (while respecting `MAX_WIDTH`), potentially creating and sorting new bins, and adaptive control when exceeding `maxBins`.

Table 5
Ingestion Rate Per Second (Ingestion Speed) of All Schemes

Scheme	Ingestion Rate values/sec
HDR	45794780
DH	42649692
DD	35058203
GB	13399434
TD	8682654
BML	1263719

4.2.3. Quantile-query speed / Latency

Query latency was measured after loading 4.32M RTT values into each scheme. We executed 100,000 mixed P50, P90, and P99 queries using System.nanoTime(). Table 6 reports total time and average latency. HDR was fastest at about 119 ns/query,

followed by TD at 121 ns/query. GB recorded about 852 ns/query, which is slower than HDR and TD but still below one microsecond. DD and DH required more traversal, while BigML was slowest at about 12.8 μs/query. Thus, HDR and TD are better for very frequent queries, while GB remains practical with readable adaptive-bin output.

Table 6
Quantile Query Latencies

Scheme	Total time (ms)	Avg time /Query (ns)	Avg time /Query (μs)
HDR	11.95	119.5	0.1195
TD	12.13	121.3	0.1213
GB	85.27	852.7	0.8527
DD	119.37	1193.7	1.1937
DH	136.49	1,364.90	1.3649
BML	1281.58	12,815.80	12.8158

4.2.4. Concluding Remarks on Percentile and Speed Tests

These results show that there is a balance between model accuracy and speed. GB provided moderate speeds for ingestion and querying but the lowest errors for P50 and P90. It also matched the best result at P99, while effectively preserving the most informative regions of the distribution. This demonstrates its strong statistical fidelity and precision for single-node streaming analytics.

HDR thrived on ingestion throughput and query latency, which makes it well suited for speed-critical applications; however, its mid-range quantile accuracy indicates limited resolution in capturing fine-grained information in the central regions. DH and DD showed balanced behavior, combining good ingestion efficiency with acceptable precision. TD had a faster query, but slower ingestion, and BML gave reasonable accuracy with poor runtime performance.

4.3. Resource Consumption Comparisons

During percentile estimation test (Section 4.2.1), the heap usage and CPU usage were monitored for all schemes for 30 minutes and sampled at 5-second intervals. Figure 4 compares JVM heap traces obtained from JVisualVM and code-based CSV logging. All traces displayed the classic "sawtooth" GC pattern: steady climb of the heap followed by steep declines because of full GC passes. Heap size always fell back to a baseline (13–17 MB), thus, no memory leaks nor did we notice any irregular allocation patterns. GC cycles occurred every 100–150 MB of allocations, with complete memory recovery after each cycle.

Table 7 summarizes average heap usage. DH and DD showed the lowest average heap usage, HDR and TD were in the mid-range, GB was slightly higher, and BigML was the highest.

Figure 5 reports JVM CPU usage from the CSV logs. Table 8 reports average CPU utilization, again showing minimal differences: BigML (0.195%) $<$ DH (0.200%) $<$ TD (0.204%) $<$ HDR (0.206%) $\approx$ DD (0.206%) $<$ GB (0.217%). We can see that CPU values differed by only about 0.02%, showing very little overhead for all schemes.

Average CPU usage remained below 0.25% for all schemes.

4.4. Interpretability of Histograms

Readable bin ranges are useful because the histogram is meant to be inspected by a user, not only queried by code [15]. GB was shown using DisplayBinsUtil, while DH,

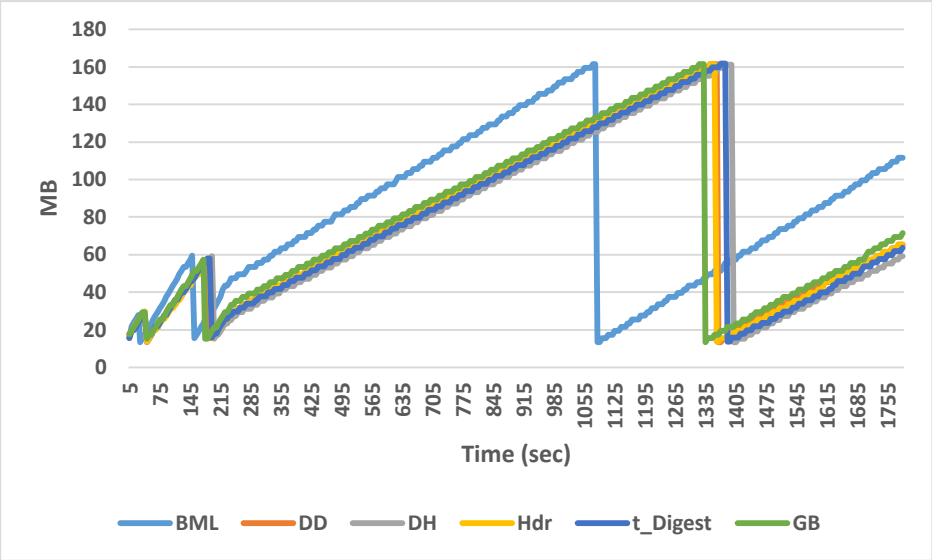


Figure 4. JVM heap usage traces for all schemes over a 30-minute run, sampled every five seconds.

Table 7
Average JVM Heap Usage of Each Scheme During the Whole Run

Scheme	Avg. JVM heap usage (MB)
DH	73.69603352
DD	73.91759777
HDR	74.44332402
TD	74.45932961
GB	74.94840782
BML	78.66148045

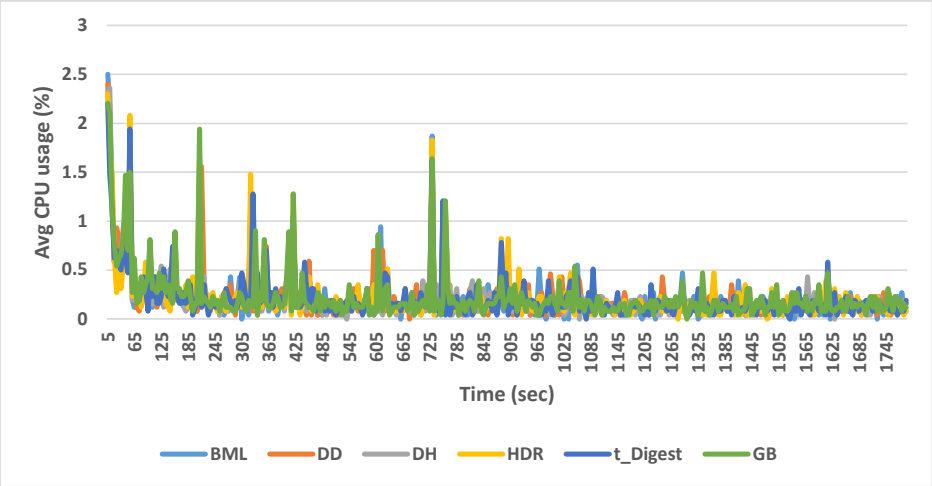


Figure 5. JVM CPU usage over time for all schemes during isolated runs.

Table 8
Average JVM CPU Usage of Schemes During the Whole Run

Scheme	Avg. CPU usage (%)
BML	0.195322129
DH	0.199553073
TD	0.204050279
HDR	0.205810056
DD	0.206498599
GB	0.21697479

HDR, and BML were shown using their native or directly exposed bin structures. The comparison used the skewed PlanetLab stream [23]. The synthetic workloads (Normal/Poisson/exponential) had similar results provided in the repository. The GB method gave a neater set of bins. The other methods often resulted in small, irregular, or uneven bins.

Since interpretability depends on human reading of bin structure, it was treated as a qualitative comparison rather than a numerical scoring experiment. We therefore treated this experiment as a native-output readability comparison, not as a statistically controlled rebinning benchmark.

4.4.1. Histograms on Highly Skewed Dataset

Figure 6 shows the histograms of all schemes generated on a highly skewed dataset [23] at an ingestion rate of 1000 values per second.

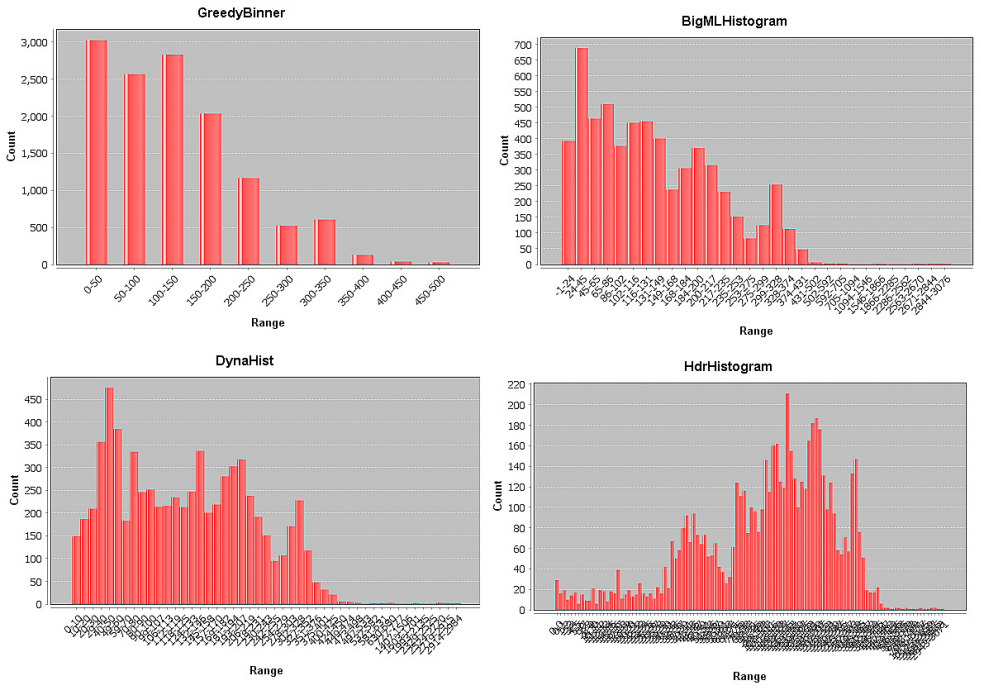


Figure 6. Native dashboard-facing histogram outputs of the evaluated schemes on the highly skewed PlanetLab RTT dataset.

4.4.1.1. Compactness Through its built-in display utility, GB summarized millions of values into about ten clean, evenly spaced dashboard-facing bins (e.g., 0–50, 50–100, ...) while its internal adaptive structure continued to adjust as data evolved.

BML generated dozens of inconsistent bins, varying from narrow to extremely wide, while DH produced irregular fragments and large jumps in width. HDR Histogram created hundreds of micro-bins near zero and coarse aggregations at the tail. Overall, GB offered the most compact yet representative summary, unlike others that either fragmented or over-aggregated data.

4.4.1.2. Semantic Coherence GB's bins aligned with natural, rounded boundaries (multiples of 50), producing readable ranges. BML's arbitrary limits (e.g., 139–148, 157–168) and DH's irregular overlaps (e.g., 29–40, 49–60) lacked intuitive meaning, while HDR's micro-ranges (7–8, 8–9) had little semantic value. GB maintained coherent, human-friendly intervals.

4.4.1.3. Human Understandability GB is easier to read on the skewed stream because its bins stay compact and use clearer range labels.

4.5. Repeated-Run Analysis on Random Seeds

Each scheme was executed five times using seeds 0–4. The workload was made non-stationary with bursts by setting it to 1000 requests/sec as a base rate, then adding 1-sec burst peaks of between 1500 and 1700 requests/sec every 10 seconds. Values were gathered from PlanetLab [23] per second. From the 5 executions, the mean relative error, standard deviation and 95% confidence interval were calculated. An example of the 10 minute workload using seed 0 is shown in Figure 7.

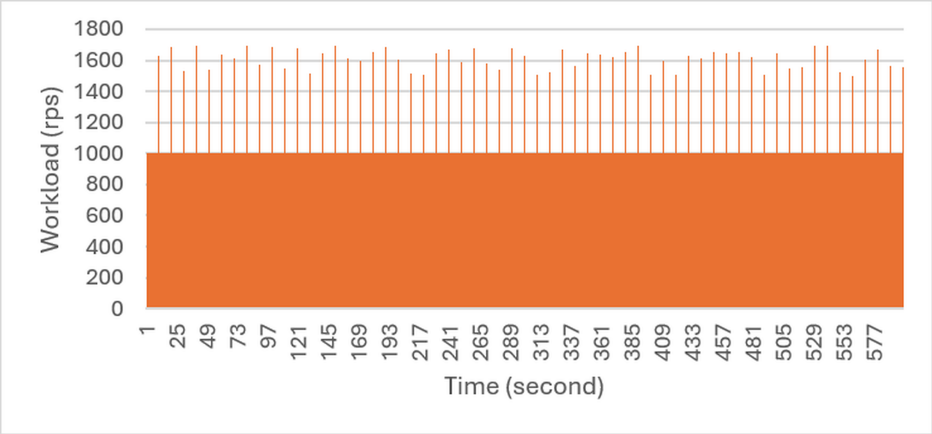


Figure 7. Producer–consumer workload using PlanetLab [23] values for the tested schemes.

In Table 9, the lowest mean relative errors were seen for GB at P50 (0.030%) and P90 (0.090%). However, it was equal to HDR at P99 (0.120%). The standard deviation of these values were low (0.007%, 0.010% and 0.011%) respectively. This indicates that there is little variation between the 5 runs.

Looking at the 95% confidence interval in Table 10 we can see that GB had a relatively low error range of [0.021, 0.039]% at P50, [0.078, 0.102]% at P90 and [0.106, 0.134]% at P99. This demonstrates that the low error observed was consistent across independent runs, and not attributable to a single random workload.

Table 9
Mean ± SD of Relative Error (%)

Scheme	P50	P90	P99
BML	0.290 ± 0.018	0.170 ± 0.014	0.300 ± 0.021
DD	0.250 ± 0.020	0.680 ± 0.045	0.480 ± 0.035
DH	0.100 ± 0.011	0.380 ± 0.025	0.240 ± 0.018
HDR	0.340 ± 0.020	0.160 ± 0.013	0.120 ± 0.010
TD	0.320 ± 0.022	0.470 ± 0.030	0.660 ± 0.042
GB	0.030 ± 0.007	0.090 ± 0.010	0.120 ± 0.011

Table 10
95% Confidence Interval of Relative Error (%)

Scheme	P50	P90	P99
BML	[0.268, 0.312]	[0.153, 0.187]	[0.274, 0.326]
DD	[0.225, 0.275]	[0.624, 0.736]	[0.436, 0.524]
DH	[0.086, 0.114]	[0.349, 0.411]	[0.218, 0.262]
HDR	[0.315, 0.365]	[0.144, 0.176]	[0.108, 0.132]
TD	[0.293, 0.347]	[0.433, 0.507]	[0.608, 0.712]
GB	[0.021, 0.039]	[0.078, 0.102]	[0.106, 0.134]

4.6. Overall Positioning of GreedyBinner

For a final quick view, we can see the overall positioning of proposed scheme in Table 11.

Table 11
Overall positioning of GreedyBinner across evaluation measures.

Evaluation dimension	GB result	Overall position
P50 relative error	0.02%	1st
P90 relative error	0.10%	1st
P99 relative error	0.11%	Joint 1st with HDR
Error stability	Best or second-best across tested quantiles	Top tier
Ingestion throughput	13.4 M values/s	Mid-range
Query latency	852.7 ns/query	Sub-microsecond, mid-range
JVM heap usage	74.95 MB	Comparable to other methods
JVM CPU usage	0.217%	Slightly highest, but negligible in absolute terms
Histogram interpretability	Compact, coherent, and readable bins	Strongest qualitative result

5. Conclusion

The GreedyBinner scheme was presented in this paper for single node streaming quantile estimation and monitoring. The algorithm maintains a fixed upper limit on the number of active bins by limiting the maximum number of bins and merging low frequency neighbor bins together after controlling the bin widths. For the skewed

latency stream, the method had the lowest relative errors for P50 and P90. For P99, the method matched the best performing approach. Additionally, the histogram ranges were interpretable and would be easily displayed on monitoring dashboards. While the speed of the algorithm was not the fastest, the ingest rate, query latency, memory and CPU usage remained practical for monitoring latency in a single node system where storage is limited. Further research will be done to apply the method to further streams. Additionally, distributed and automated parameter approaches will be considered.

Declarations

Data Availability Statement

The GreedyBinner project: URL withheld for double-blind review.

Experimental CSV archive: DOI withheld for double-blind review.

The PlanetLab latency dataset used in the evaluation: `https://github.com/uofa-rzhu3/NetLatency-Data`

Funding

The authors did not receive support from any organization for the submitted work.

References

[1] Agarwal P.K., Cormode G., Huang Z., Phillips J.M., Wei Z., Yi K.: Mergeable summaries. In: *ACM Transactions on Database Systems*, vol. 38(4), pp. 26:1–26:28, 2013. URL `http://dx.doi.org/10.1145/2500128`.

[2] BigML: Streaming histograms. BigML Documentation, 2021. URL `https://github.com/bigmlcom/histogram`. `https://bigml.com`; Accessed 27 June 2026.

[3] Cormode G., Karnin Z., Liberty E., Thaler J., Veselý P.: Relative error streaming quantiles. In: *Journal of the ACM*, vol. 70(6), pp. 1–34, 2023. URL `http://dx.doi.org/10.1145/3617891`.

[4] Correll M., Li M., Kindlmann G., Scheidegger C.: Looks good to me: Visualizations as sanity checks. In: *IEEE Transactions on Visualization and Computer Graphics*, vol. 25(1), pp. 830–839, 2019. URL `http://dx.doi.org/10.1109/TVCG.2018.2864907`.

[5] Dean J., Barroso L.A.: The tail at scale. In: *Communications of the ACM*, vol. 56(2), pp. 74–80, 2013. URL `http://dx.doi.org/10.1145/2408776.2408794`.

[6] Dunning T., Ertl O.: Computing extremely accurate quantiles using t-digests. arXiv preprint arXiv:1902.04023, 2019. URL `http://dx.doi.org/10.48550/arXiv.1902.04023`.

[7] Dynatrace OSS: DynaHist: A dynamic histogram library for Java, release 1.4. GitHub repository, 2023. URL `https://github.com/dynatrace-oss/dynahist`. Accessed 27 June 2026.

[8] Epicoco I., Melle C., Cafaro M., Pulimeno M., Morleo G.: UDDSketch: Accurate tracking of quantiles in data streams. In: *IEEE Access*, vol. 8, pp. 147604–147617, 2020. URL `http://dx.doi.org/10.1109/ACCESS.2020.3015599`.

[9] Greenwald M., Khanna S.: Space-efficient online computation of quantile summaries. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 58–66. Santa Barbara, CA, USA, 2001. URL `http://dx.doi.org/10.1145/375663.375670`.

[10] Hammer H.L., Yazidi A., Riegler M.A., et al.: Efficient quantile tracking using an oracle. In: *Applied Intelligence*, vol. 53, pp. 289–300, 2023. URL http://dx.doi.org/10.1007/s10489-022-03489-1.

[11] Hartmann H., Schlossnagle T.: Circllhist: A log-linear histogram data structure for IT infrastructure monitoring. arXiv preprint arXiv:2001.06561, 2020. URL http://dx.doi.org/10.48550/arXiv.2001.06561.

[12] Łukasiewicz A., Tětek J., Veselý P.: SplineSketch: Even more accurate quantiles with error guarantees. arXiv preprint arXiv:2504.01206, 2025. URL http://dx.doi.org/10.48550/arXiv.2504.01206.

[13] Masson C., Rim J.E., Lee H.K.: DDSketch: A fast and fully mergeable quantile sketch with relative-error guarantees. In: *Proceedings of the VLDB Endowment*, vol. 12(12), pp. 2195–2205, 2019. URL http://dx.doi.org/10.14778/3352063.3352135.

[14] Oracle: Java Platform, Standard Edition Java Management Extensions Guide. Oracle Corporation. URL https://docs.oracle.com/en/java/javase/21/jmx/java-management-extensions-guide.pdf. Accessed 10 October 2026.

[15] Sahann R., Möller T., Schmidt J.: Histogram binning revisited with a focus on human perception. In: *Proceedings of the 2021 IEEE Visualization Conference (VIS)*, pp. 116–120. 2021. URL http://dx.doi.org/10.1109/VIS49827.2021.9623240.

[16] Sigelman B.H., Barroso L.A., Burrows M., Stephenson P., Plakal M., Beaver D., Jaspan S., Shanbhag C.: Dapper, a large-scale distributed systems tracing infrastructure. Tech. Rep. dapper-2010-1, Google, 2010. URL https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/. Accessed 27 June 2026.

[17] Sohana S., Pourmajidi W., Steinbacher J., Miranskyy A.: CloudHeatMap: Heatmap-based monitoring for large-scale cloud systems. arXiv preprint arXiv:2410.21092, 2024. URL http://dx.doi.org/10.48550/arXiv.2410.21092.

[18] Sriraman A., Liu S., Gunbay S., Su S., Wenisch T.F.: Deconstructing the tail at scale effect across network protocols. arXiv preprint arXiv:1701.03100, 2017. URL http://dx.doi.org/10.48550/arXiv.1701.03100.

[19] Sulewski P., Stępień R., Jędrzejewski P., Zielińska K.: Equal-bin-width histogram versus equal-bin-count histogram. In: *Journal of Applied Statistics*, vol. 48(11), pp. 2092–2111, 2021. URL http://dx.doi.org/10.1080/02664763.2020.1784853.

[20] Tene G.: HdrHistogram: A high dynamic range histogram. http://hdrhistogram.org. Accessed 27 June 2026.

[21] VisualVM: VisualVM—All-in-One Java Monitoring and Troubleshooting Tool. VisualVM Project. URL https://visualvm.github.io/. Accessed 10 October 2026.

[22] Wand M.P.: Data-based choice of histogram bin width. In: *The American Statistician*, vol. 51(1), pp. 59–64, 1997. URL http://dx.doi.org/10.1080/00031305.1997.10473591.

[23] Zhu R.: NetLatency-Data: Public network latency datasets gathered from PlanetLab and Seattle. GitHub repository. URL https://github.com/uofa-rzhu3/NetLatency-Data. Accessed 27 June 2026.

Affiliations

Faisal Bahadur
Department of CS&IT, Hazara University, Mansehra, 21300, Pakistan, faisal@hu.edu.pk

Asim Shahzad
Department of Computer Science, Abbottabad University of Science and Technology, Abbottabad, 222020, Pakistan, asimshahzad@aust.edu.pk